

Paper 3: Fixture-Driven Research Harness for Falsifiable O-RAN Remediation Claims

David Kypuros

Red Hat

dkypuros@redhat.com

31 May 2026

Abstract

Paper 3 in this five-paper series describes the fixture-driven research harness that makes covered v0 claims falsifiable. Agentic-AI-for-networks papers frequently make architectural claims that are difficult to falsify because the implementation is proprietary, partially specified, or not runnable independently. We describe the research engineering methodology behind a closed-loop Day-2 remediation harness for O-RAN Cloud RAN that takes the opposite posture: every covered v0 implementation claim anchors to a file or fixture, every authored harness or scenario YAML/JSON file carries citation metadata, and a verify gate produces a binary pass/fail result a reviewer can reproduce against a cited repository state. The methodology has four anchoring layers (citation headers, JSON Schema contracts, a bidirectional standards-lineage index, and end-to-end fixture replay) and an explicit stub-boundary discipline that names which components carry the paper’s claims and which exist only to isolate the real components from external dependencies. Full ten-check reproduction requires jsonschema; without it, schema validation is reported as skipped. The LLM-neutral design is enforced structurally rather than by convention, which preserves determinism in the verify gate. We argue this style of research substrate produces bounded, falsifiable outputs independent of LLM provider, API availability, or model-version drift.

1 Introduction

For a software harness to serve as a research substrate (as opposed to a product prototype), it should produce bounded claims that a reviewer can falsify mechanically. Three properties matter: covered v0 implementation claims are anchored to specific files, the substrate runs reproducibly without external dependencies, and stub boundaries are declared rather than hidden. The dominant pattern in agentic-AI-for-networks publications fails one or more of these properties. Implementations are described in prose without file-level anchors; the runtime depends on a live cloud, a live cluster, or a particular API key; and the stub boundary (the line between code under test and the systems being mocked) is implicit.

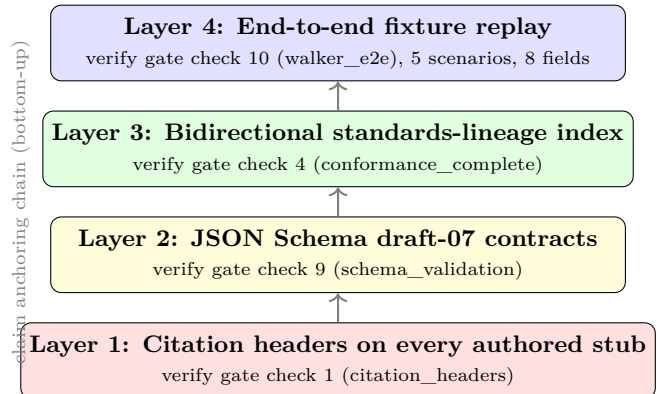


Figure 1: Four-layer claim anchoring system (read bottom-up). Each layer is mechanically enforced by a specific check of the verify gate when optional dependencies are installed. Layer 1 turns documentation into a repository contract; Layer 4 turns covered v0 routing, action, and reversibility claims into mechanically falsifiable assertions against committed expected outputs.

A prior multivendor diagnostic publication [1] established a fault-localization substrate for Cloud RAN troubleshooting. The closed-loop extension we describe in companion papers takes the same posture: the research artifact is the runnable open-source repository, the architectural narrative is anchored at file plus line, and the verify gate is the falsification mechanism. This paper documents the methodology by which the three substrate properties are achieved. The repository under discussion is open-source under Apache-2.0 at <https://github.com/dkypuros/oran-agent-harness>.

The remainder of the paper is organized as follows. Section 2 documents the four-layer claim anchoring system. Section 3 describes the verify gate’s ten possible checks and the failure mode each one guards. Section 4 describes the stub-boundary discipline. Section 5 describes the LLM-neutral design as a research methodology requirement, not only a production concern. Section 6 compares the approach to adjacent research engineering practices. Section 7 states the reproducibility entry point. Section 10 concludes.

2 The Claim Anchoring System

Four layers of anchoring carry the load. Figure 1 shows the layered hierarchy.

2.1 Layer 1: Citation headers on every stub

Every YAML and JSON file authored under `harness/` and `scenarios/` opens with a `_conforms_to` or equivalent block naming the spec lineage, the spec section, the spec version, and the bibliography reference numbers. The verify gate's first check enforces this: any new file added without a citation header fails the gate. The mechanism turns documentation into a repository contract. Adding a routing rule, a schema, a guardrail, or a scenario fixture without naming its lineage is a build failure, not a documentation oversight.

2.2 Layer 2: JSON Schema draft-07 contracts

Five harness-authored schemas define the shapes the pipeline produces and consumes: `FaultPayload.json`, `RCA.json`, `RemediationProposal.json`, `AuditEvent.json`, and `ReversibilityProfile.json`. With `jsonschema` installed, each scenario's `fault_payload.json` validates against `FaultPayload.json`; each scenario's embedded remediation object validates against `RemediationProposal.json`; and each embedded reversibility profile validates against `ReversibilityProfile.json`. The gate's ninth check runs those validations. A claim like "the audit event always carries a populated reversibility profile" stops being a prose assertion and becomes a schema-shape validation outcome. The full TMF688 audit envelope remains a harness-authored shape claim, not a standards-body certification.

2.3 Layer 3: Bidirectional standards-lineage index

A single file `harness/conformance.md` is a two-column table mapping every authored artifact in the harness tree to its spec lineage and bibliography references. The gate's fourth check confirms both directions: every file under `harness/` or `scenarios/` appears in the table, and every row in the table points at a file that exists. The mechanism prevents the index from rotting silently as the codebase changes. A new file without an entry fails the gate; an entry pointing at a deleted file fails the gate. This is traceability of standards-lineage metadata, not formal conformance certification.

2.4 Layer 4: End-to-end fixture replay

The gate's tenth check runs the deterministic walker as a subprocess against all five committed scenario fault payloads. For each scenario, the actual audit event output is compared against the committed expected

audit event on eight material comparisons: seven named fields (`targetLayer`, `taxonomyMatch`, `actionType`, `ocloudInternalPath`, `actionPayloadRef`, `dryRun`, `requiresHumanApproval`) plus the `reversibility_profile.confidence_in_reversibility` sub-field comparison. Any divergence on any comparison fails the gate. The mechanism turns selected v0 claims about routing decisions, action types, and the seven-field reversibility contract into mechanically falsifiable assertions.

3 The Verify Gate

The gate is a single Python script at `scripts/verify.py`. With `PyYAML` and `jsonschema` installed, it runs ten checks in sequence and exits 0 on 10 of 10 pass, non-zero with specific failure rows otherwise. Without `jsonschema`, check 9 is reported as skipped and the reduced gate reports the executed checks. Figure 2 shows the three-phase pipeline. Table 1 maps each check to the bounded claim it supports, the evidence inspected, and the limit that remains outside the v0 claim. Each check guards a specific failure mode.

Check 1, citation_headers. Guards against undocumented stubs. A new YAML or JSON file added without a `_conforms_to` block fails the check.

Check 2, json_parse. Guards against syntax rot in any JSON file across the public tree.

Check 3, yaml_parse. Same for YAML files.

Check 4, conformance_complete. Bidirectional standards-lineage index integrity (Layer 3 above).

Check 5, em_dashes. Guards against a specific authorship tell that the author has committed to eliminating from published copy. The check reports zero permitted occurrences of the em dash codepoint (U+2014) in the public tree.

Check 6, no_leakage. Guards against accidentally committing `.env` files, `.local/` content, credentials, or large local demo artifacts. Runs against `git ls-files` so it catches anything the working tree might hide.

Check 7, readme_structure. Guards against the README drifting from the three stated repository goals. Enforces line-count bounds and the presence of the three-goal lede.

Check 8, file_counts. Guards against silent file additions or deletions that change the harness surface without updating the index. The count is asserted at known totals.

Check 9, schema_validation. Scenario fault payloads and embedded remediation plus reversibility sub-objects validate against the harness-authored JSON Schemas (Layer 2 above). The check is required for a full ten-check reproduction and auto-skips if the `jsonschema` package is not installed, with a warning.

Check 10, walker_e2e. The falsifiability check (Layer 4 above). Spawns the walker as a subprocess, captures the audit event, compares to the committed expected output on material fields.

Run-time of the entire gate against the reference repository state is approximately five seconds on a developer-

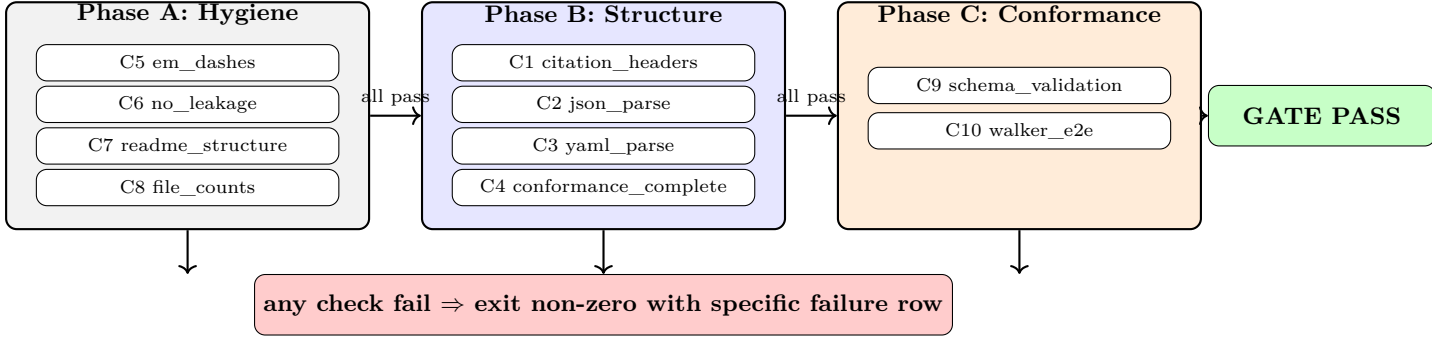


Figure 2: Ten-check verify gate as a three-phase pipeline when optional schema validation is available. Phase A enforces hygiene (em-dash audit, leakage guard, README structure, file counts). Phase B enforces structural integrity (citation headers, JSON and YAML parse, standards-lineage index). Phase C enforces schema-shape and fixture-replay integrity (JSON Schema validation, end-to-end walker replay). Any executed check failure exits the gate non-zero with a specific failure row.

grade workstation. The dependency surface is Python 3.8 or later, PyYAML, and (optionally) jsonschema. No cloud account, no live cluster, no LLM API key is required for the deterministic gate.

4 Stub Boundary Discipline

The harness is explicit about what is real and what is stubbed. The methodology argument is that the components carrying the paper’s claims must be real; the components that exist only to isolate them from external dependencies should be stubbed cleanly.

Real components. The `harness/runtime/router.py` module performs the deterministic taxonomy lookup against the authored `harness/taxonomy.yaml`. It is exercised end-to-end in check 10 of the verify gate. The `harness/runtime/guardrail.py` module evaluates the five-subcheck policy guardrail against the authored `harness/guardrails.yaml`. Both modules’ deterministic behavior is the source of the paper’s claims about routing and gating. They are real Python code executing real YAML inputs.

The repository also ships a real Python function call that exercises an O2-IMS-shaped adapter boundary from the architecture diagram: `5G_0-RAN_SIM/oam/o2ims_stub.py`’s `deploy_request` function is invoked by the router on every infrastructure-layer route, and the response envelope is attached to the audit. The stub is shape-only (no network call, no real reconcile loop), but the function call is real, and the test `test_o2ims_dispatch_attached_on_down_route` asserts that the response shape is contract-correct.

Stubbed components. The Agentic Gateway, the four MCP servers (`mcp-platform`, `mcp-ran`, `mcp-hardware`, `mcp-ocloud`), the three domain agents (Platform, RAN, Hardware), and the Digital Twin substrate are stubbed. The walker’s module docstring names them directly. The stub boundary is declared, not hidden. Each scenario’s canned values (RCA timestamp, contributing signals, action tar-

get, blast radius, sandbox verdict, SMO dispatch outcome, O2 IMS dispatch outcome) live in a single file at `harness/runtime/scenario_stubs.json`, which serves as the explicit single source of truth for stubbed data.

This design decision matters for the research substrate property. The real components (router, guardrail, `o2ims` function call) are the ones that carry the paper’s architectural claims, and they are exercised against authored YAML and JSON inputs the verify gate validates. The stub components are wrappers that isolate the real components from external dependencies (a live MCP server, a live LLM endpoint, a live Twin cluster) so the gate can run without network access, credentials, or vendor-specific tooling. A reviewer can change the stubbed inputs and observe the real components’ deterministic response.

5 The LLM-Neutral Substrate as a Research Property

The harness’s third declared contribution is an LLM-neutral substrate. The motivation is partly production-pragmatic (avoid lock-in to a specific provider) but the research methodology value is stronger.

If the runtime embedded a specific LLM provider’s SDK or a specific model identifier, the verify gate’s tenth check (walker end-to-end) would have nondeterministic output. Different reviewers running the gate against the same repository state at the same git SHA would see different outcomes depending on provider availability, model version, or even response-time jitter. The research substrate property would collapse.

The harness enforces LLM neutrality structurally rather than by convention. The routing rule’s ambiguous path raises `NotImplementedError` in the default code path, preserving determinism in the verify gate. The live LLM path is gated behind an environment variable (`ORAN_LLM_MODE=live`) and a separate test (`test_route_ambiguous_with_llm_mode_live`) that skips by default to keep the gate result independent. Provider reachability for Anthropic,

Check	Claim bounded by the gate	Evidence inspected	Limit
C1	Authored harness and scenario YAML/JSON files carry standards-lineage metadata	<code>_conforms_to</code> keys or citation headers under <code>harness/</code> and <code>scenarios/</code>	Metadata presence, not standards-body certification
C2	Public JSON files are syntactically parseable	JSON parser over the public tree	Syntax only, not semantic correctness
C3	Public YAML files are syntactically parseable	PyYAML parser over the public tree	Syntax only, not policy correctness
C4	The standards-lineage index is bidirectionally complete	<code>harness/conformance.md</code> rows compared against files under <code>harness/</code> and <code>scenarios/</code>	Only indexed repository artifacts are covered
C5	The public tree satisfies the manuscript’s em-dash hygiene rule	Unicode scan for U+2014	Style hygiene only
C6	Tracked files do not include the configured leakage patterns	<code>git ls-files</code> checked for forbidden paths and PDFs outside allowed paper outputs	Not a general secret scanner
C7	The README keeps the expected public entry structure	Line-count bounds and three required lede strings	Structure check, not full documentation audit
C8	The public harness surface has not silently expanded or shrunk	Fixed counts for <code>harness/</code> and skill markdown files	Intentional scope growth requires updating the gate
C9	Scenario payloads and embedded remediation objects match harness-authored schemas	<code>jsonschema</code> validation of fault payloads, remediation objects, and reversibility profiles	Optional dependency; schemas are harness-authored research contracts
C10	The deterministic walker reproduces covered v0 routing, action, and reversibility outcomes	Subprocess replay of five committed scenarios against expected audit fixtures on eight material comparisons	Partial scenario coverage over stubbed partner-system inputs

Table 1: Claims proven by the verify gate. Each row states the bounded claim, the evidence the script inspects, and the limit that should remain visible in the manuscript. The table prevents the phrase "verify gate pass" from being read as live O-RAN, TM Forum, Kubernetes, Redfish, or LLM interop.

the OpenAI API, or an OpenAI-compatible on-premises vLLM endpoint is tested separately by `scripts/test_llm_live_path.py`; it is not part of the committed scenario replay. The structural enforcement means a reviewer cannot accidentally make the gate provider-dependent.

The LLM is therefore not load-bearing on the routing decision the verify gate proves. This is the right division of labor for a research substrate: the deterministic claims are reproducible at compile time, the LLM-assisted claims are gated behind a separate execution mode that does not interfere with the deterministic gate. A v1 deployment that wants real LLM behavior on the ambiguous path can light up the env-gated seam without touching the deterministic core.

6 Comparison to Adjacent Research Practices

Three adjacent research engineering practices illustrate the niche the harness occupies.

Unit tests without spec-anchored fixtures. Most research code ships with tests that confirm internal consistency (a function returns what its signature promises) but not external spec lineage or shape compatibility. The harness’s check 10 binds the test output to committed expected audit fixtures, and those fixtures themselves validate against harness-authored, spec-cited JSON Schemas when `jsonschema` is installed. A change

that breaks a covered v0 routing, action, or reversibility claim breaks the gate.

Integration tests against live services. Some research projects integration-test against a live cluster or API. This is honest about end-to-end behavior but defeats reproducibility: a reviewer without the live service cannot rerun the test. The harness preserves end-to-end coverage through fixture replay against committed canned inputs, which a reviewer can re-execute on a clean clone.

Pure simulation. Some research uses simulators that never touch real code paths through standardized interfaces. The harness occupies a middle position: the deterministic routing and guardrail logic are real Python executing real YAML; the simulated component is the partner system (the O-Cloud Manager, the partner SMO, the BMC) which is stubbed at the shape-correct envelope boundary. A v1 deployment can swap the stubs for live targets behind the same envelope.

7 Reproducibility Entry Point

A reader who wants to reproduce the full harness verify gate runs four commands.

```
git clone \
  https://github.com/dkypuros/oran-agent-harness
cd oran-agent-harness
pip install pyyaml jsonschema
python3 scripts/verify.py
```


That is the complete reproducibility claim for the deterministic gate. No cloud account, no live cluster, and no LLM API key are required. The reference manuscript state is repository commit `a2686f42755ac9a0be27f3f97c5d142d6315047a`, dated 2026-05-31. At that referenced state, a clean clone with PyYAML and jsonschema installed is the state intended by the paper’s SUMMARY: 10/10 checks passed and OVERALL: PASS claim. In reduced environments, `scripts/verify.py` reports check 9 as skipped and prints the executed-check denominator; dirty local working trees can fail hygiene checks independently of the referenced manuscript state.

Beyond the gate, the repository ships two lab postures. The macOS posture at `macbook_lab/` uses Docker Desktop and a multi-container compose file to stand up a fuller stack with HTTP wrappers around the platform stubs. The Linux posture at `linux_lab/` uses podman 5.4.1 on Fedora 42 with the same compose file. A verified run document at `linux_lab/verified_runs/2026-05-31_fedora42_podman.md` captures the live cross-runtime verification.

These lab postures are reviewer aids, not replacements for the deterministic proof. They expose the same fixtures through browser and chat surfaces, while the publication-grade falsification claim remains the clone, install, and `scripts/verify.py` path above.

8 Reviewer-Operability Instrumentation

The deterministic verify gate is the publication-grade falsification mechanism. The repository also ships a separate operator-facing inspection surface at `omc-skills/oran-discover/`. This surface is methodological instrumentation: it helps a reviewer or operator inspect the same state the papers discuss, but it is not the mechanism that proves the covered v0 claims.

The bundle consists of six read-only probes plus one orchestrator: `/oran-discover:ptp` for PTP and Cloud Notification state, `/oran-discover:metal3` for BMO and HostFirmwareComponents state, `/oran-discover:redfish` for BMC task state, `/oran-discover:smo` for TMF921 companion-intent state, `/oran-discover:taxonomy` for the 20-entry routing taxonomy, `/oran-discover:guardrail` for policy and crisis-mode state, and `/oran-discover:plan` as the pre-flight walk over the other six. The conformance index at `omc-skills/oran-discover/conformance.md` maps each skill to its live wrapper, static fallback file, and reference lineage.

The MacBook dashboard adds a reviewer-operable front door around that bundle. A reviewer can open the simple two-loop demo, request `/commands` to list the scoped slash-command surface, run `/oran-discover:plan` as a pre-flight survey, switch among `D_phc_drift_hw_only`, `E_nic_firmware_update`, and `E_with_smo_reject`, and attach a `harness-walker` sidecar via `/run/{scenario}` to display the live AuditEvent.

With an assistant key configured, the chat harness can then summarize what the event proves and what remains blocked behind the operator. That explanation is convenience instrumentation; it may depend on the local lab and model configuration, whereas the deterministic verify gate does not.

A separate check at `scripts/test_oran_discover_skills.py` verifies that the seven skill markdown files have the required structure, that their static source files exist, and, when the MacBook lab wrappers are running, that the live endpoints return the expected JSON keys. This complements `scripts/verify.py`. The verify gate proves bounded implementation claims through schemas and fixture replay; the discovery-skill check proves that the human-facing read-before-write surface is wired and documented.

9 Honest Scope

The methodology produces falsifiable claims for the bounded set of behaviors the verify gate covers. Several explicit limits.

Fixtures are canned. Scenario inputs and the partner-system stubbed responses do not come from a running O-Cloud or a real `linuxptp` daemon. A real cluster could produce inputs that fail validation against the JSON Schemas; that would be informative but is out of scope for the gate.

Schemas are harness-authored. The JSON Schema draft-07 contracts at `harness/schemas/` are the harness’s own approximations of the spec envelopes. The schemas are not published by O-RAN, TM Forum, or any other standards body. A real spec-conformance test would use the standards bodies’ own test suites (for example the O-RAN SC compliance test suite [2]); that is a v1 work item.

Scenario coverage is partial. The five committed scenarios cover the primary delivery path branches (MCO, KMM, Metal3, dual-route accepted, dual-route SMO-rejected) but 15 of the 20 taxonomy entries have no walkthrough fixture. A future research artifact could claim broader coverage once those fixtures exist.

Some signals are stubbed values. EvalOps confidence telemetry, Agentic Recoverability validation history, the full Digital Twin convergence envelope, and partner-system responses are described in the architectural narrative and the schemas but populated as stubbed values in `scenario_stubs.json` for v0. Live LLM provider reachability is a separate optional test, not a committed scenario fixture. Their real-time operation is a v1 deliverable.

10 Conclusion

This paper has described the research engineering methodology behind a closed-loop Day-2 remediation harness for O-RAN Cloud RAN. Four anchoring layers (citation headers, JSON Schema contracts, a bidirectional standards-lineage index, fixture replay) and a

verify gate make covered v0 implementation claims mechanically falsifiable. The stub boundary is declared, not hidden, and the components carrying the paper’s claims are real code exercised against authored inputs. LLM neutrality is enforced structurally, which preserves verify-gate determinism. The combination produces a research substrate that any reviewer can reproduce on a clean clone in under five seconds when dependencies are installed. We argued this is a meaningful improvement over agentic-AI publications that ship prose architectural claims without spec-anchored falsification mechanisms. v0 limits are declared; the v1 path is named.

Acknowledgements

The methodology builds on a prior multivendor diagnostic substrate co-authored with S. Kiani Mehr, N. Korati Prasanna, M. Vazquez Cebrian, S. Srikanthan, and P. Venkatesh [1]. Source code is Apache-2.0 licensed at <https://github.com/dkypuros/oran-agent-harness>.

References

- [1] S. Kiani Mehr, N. Korati Prasanna, D. Kypuros, M. Vazquez Cebrian, S. Srikanthan, and P. Venkatesh. Multivendor agentic AI solution for cloud RAN troubleshooting. Ericsson Technology Blog. <https://www.ericsson.com/en/blog/2026/5/multivendor-agentic-ai-solution-for-cloud-ran>, 2026. Published 11 May 2026. Accessed 14 May 2026.
- [2] O-RAN Software Community. O2 IMS compliance test suite. <https://github.com/o-ran-sc/it-test>, 2026. Accessed 14 May 2026.